FUNÇÕES

Funções pré-definidas, criando funções, escopo de funções, função como argumento

Por que criar ou usar funções?

FUNÇÕES PRÉ-DEFINIDAS

- As linguagens de programação têm à sua disposição várias funções pré-definidas:
- EXEMPLO (Linguagem *Python*)
  - input()
  - print()
  - math.sqrt(16) (utilizando *import math*)
  - abs(-3)



FUNÇÕES PRÉ-DEFINIDAS

- As Funções Pré Definidas podem ser usadas diretamente em expressões:

- Exemplo:

```
h= math.sqrt (a + y**2 + 2 * abs(math.sin(y) ))
```



Exemplos

CRIANDO FUNÇÕES

- Permitem modularizar um programa.
- Facilitam o uso da abordagem dividir e conquistar no desenvolvimento de programas.
- Facilitam a reusabilidade de códigos.

CRIANDO FUNÇÕES

```
#declaração da função
def <nome>(<argumentos>):
    """
    Descreve input
    Descreve output
    """
    instrução
    instrução
    instrução
    instrução

    return <valorRetornado>

#chamada da função
variavel=<nome>(<arg1,arg2,...,argN)
```

CRIANDO FUNÇÕES

```
#declaração da função
def <nome>(<argumentos>):
    """
    Des
    Descr
    """
    instrução
    instrução
    instrução
    instrução

    return <valorRetornado>

#chamada da função
variavel=<nome>(<arg1,arg2,...,argN)
```

Palavra reservada

CRIANDO FUNÇÕES

```
#declaração da função
def <nome>(<argumentos>):
        """
    Descreve input
    Descreve output
    """
    instrução
    instrução
    instrução
    instrução

    return <valorRetornado>

#chamada da função
variavel=<nome>(<arg1,arg2,...,argN)
```

Parâmetros formais

CRIANDO FUNÇÕES

```
#declaração da função
def <nome>(<argumentos>):
    """
    Descreve input
    Descreve output
    """
    instrução
    instrução
    instrução
    instrução

    return <valorRetornado>

#chamada da função
variavel=<nome>(<arg1,arg2,...,argN)
```

docstring (opcional!!)

CRIANDO FUNÇÕES

```
#declaração da função
def <nome>(<argumentos>):
    """
    Descreve input
    Descreve output
    """
    instrução
    instrução
    instrução
    instrução

    return <valorRetornado>

#chamada da função
variavel=<nome>(<arg1,arg2,...,argN)
```

Corpo da função

CRIANDO FUNÇÕES

```
#declaração da função
def <nome>(<argumentos>):
    """
    Descreve input
    Descreve output
    """
    instrução
    instrução
    instrução
    instrução

    return <valorRetornado>

#chamada da função
variavel=<nome>(<arg1,arg2,...,argN)
```

Palavra reservada

CRIANDO FUNÇÕES

```
#declaração da função
def <nome>(<argumentos>):
    """
    Descreve input
    Descreve output
    """
    instrução
    instrução
    instrução
    instrução

    return <valorRetornado>

#chamada da função
variavel=<nome>(<arg1,arg2,...,argN)
```

CRIANDO FUNÇÕES

```
#declaração da função
def <nome>(<argumentos>):
        """
    Descreve input
    Descreve output
    """
    instrução
    instrução
    instrução
    instrução

    return <valorRetornado>

#chamada da função
variavel=<nome>(<arg1,arg2,...,argN)
```

Parâmetros reais

CRIANDO FUNÇÕES

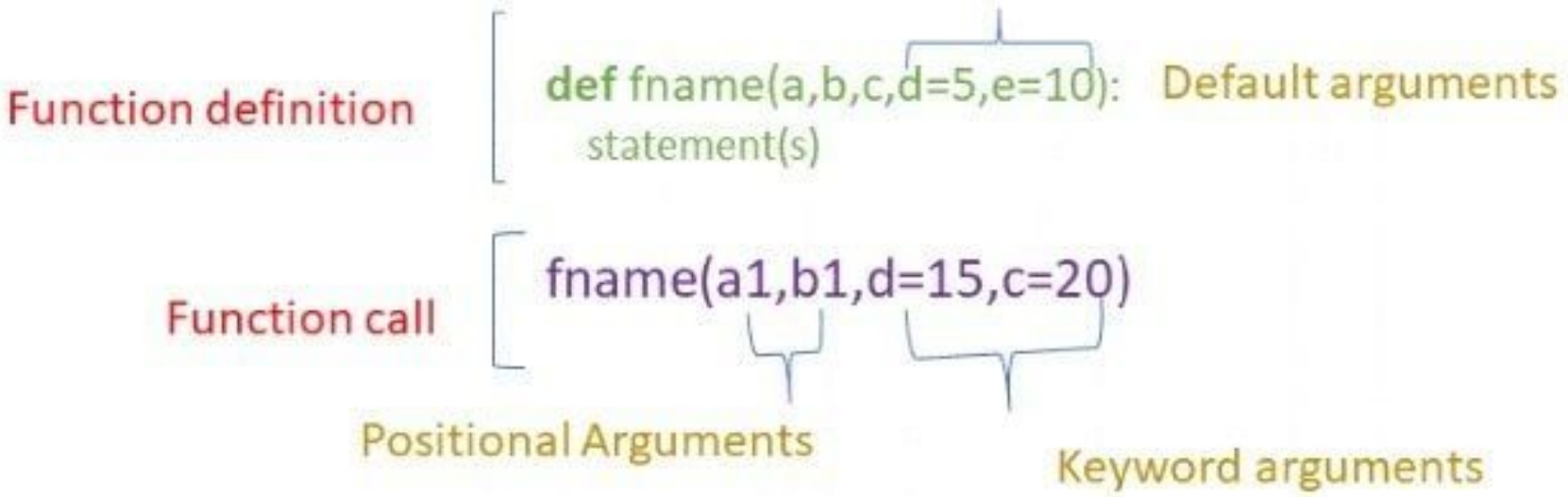
- O comando **return** é responsável por encerrar a execução da função e retornar o valor desejado.

- Uma função sem o comando return retorna None que indica a ausência de valor retornado.

Exemplos

Argumentos

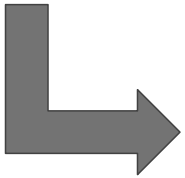
- **Argumentos posicionais** seguem a ordem de entrada dos parâmetros na função.
- **Keyword arguments (kwargs)** são identificados pelos nomes dos parâmetros.
  - Usados juntamente com valores estabelecidos (*default*) para os parâmetros.



Argumentos

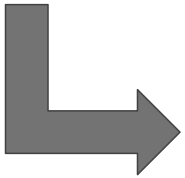
```python
def print_name(first_name,last_name,reverse):
    if reverse:
        print(f'{last_name}, {first_name}')
    else:
        print(f'{first_name}, {last_name}')

print_name('João','Silva',False)
print_name('João','Silva',reverse=False)
print_name('João',last_name='Silva',reverse=False)
print_name(last_name='Silva',first_name='João',reverse=False)
```



```
João, Silva
João, Silva
João, Silva
João, Silva
```

```python
print_name('João',last_name='Silva',False)
```

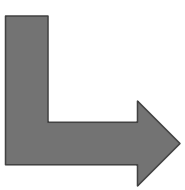


```
  File "/tmp/ipython-input-3478095720.py", line 12
    print_name('João',last_name='Silva',False)
                                        ^
SyntaxError: positional argument follows keyword argument
```

Argumentos

```python
def print_name(first_name,last_name,reverse=False):
    if reverse:
        print(f'{last_name}, {first_name}')
    else:
        print(f'{first_name}, {last_name}')

print_name('João','Silva')
print_name('Claudio','Toledo')
print_name('João','Silva',True)
print_name('João','Silva',reverse=True)
print_name(reverse=True, last_name='Silva',first_name='João')
```

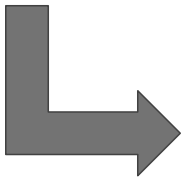


```
João, Silva
Claudio, Toledo
Silva, João
Silva, João
Silva, João
```

Argumentos

```
def print_name(first_name,last_name,reverse=False):
```

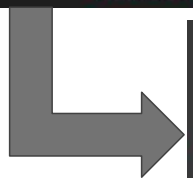
```
print_name('João',last_name='Silva')
print_name(last_name='Silva',first_name='João')
```



```
João, Silva
João, Silva
```

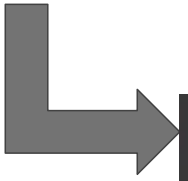
```
def print_name(first_name,last_name,reverse):
```

```
print_name('João',last_name='Silva',False)
```



```
  File "/tmp/ipython-input-3478095720.py", line 12
    print_name('João',last_name='Silva',False)
                                        ^
SyntaxError: positional argument follows keyword argument
```

```
print_name('João',last_name='Silva')
```

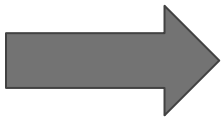


```
TypeError: print_name() missing 1 required positional argument: 'reverse'
```

Argumentos

- Funções podem gerenciar quantidades variadas de argumentos:
  - min(1,2,3)
  - min(1,2,3,4,5)
- Utiliza-se * como operador para indicar um número variado de argumentos posicionais na definição da função.

```
1   def mean(*args):
2     #Retorna a média dos valores do argumento
3     soma=0
4     for x in args:
5       soma += x
6     return soma/len(args)
7
8   print(mean(1))
9   print(mean(1,2,3))
10  print(mean(1,2,3,4,5,6))
11  print(mean(1,2,3,4,5,6,7,8))
12  print(mean(1,2,3,4,5,6,7,8,9,10))
```



```
1.0
2.0
3.5
4.5
5.5
```

Argumentos

- Os argumentos são armazenados em uma tupla.
- Podemos usar funções como map() e filter() já que * torna a variável iterável.

```python
def dobrar(x):
    """Dobrar o valor de x."""
    return x * 2

def eh_par(x):
    """Verificar se x é par."""
    return x % 2 == 0

def processar_lista(*args):
    """Processa uma lista de números usando map e filter."""
    lista = list(args)
    # Usando map para dobrar cada elemento
    lista_dobrada = list(map(dobrar, lista))
    print(f'Lista original: {lista}')
    print(f'Lista dobrada (map): {lista_dobrada}')

    # Usando filter para obter apenas os elementos
    lista_pares = list(filter(eh_par, lista))
    print(f'Lista original: {lista}')
    print(f'Lista de pares (filter): {lista_pares}

# Exemplos de uso
processar_lista(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
processar_lista(10, 20, 30, 40, 50)
```

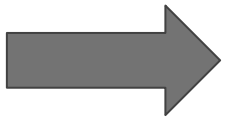
```
Lista original: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Lista dobrada (map): [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
Lista original: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Lista de pares (filter): [2, 4, 6, 8, 10]
Lista original: [10, 20, 30, 40, 50]
Lista dobrada (map): [20, 40, 60, 80, 100]
Lista original: [10, 20, 30, 40, 50]
Lista de pares (filter): [10, 20, 30, 40, 50]
```

Argumentos

- A sintaxe especial **kwargs em definições de funções é usada para passar uma lista de argumentos de comprimento variável.
- O **kwargs coleta todos os keyword arguments adicionais passados para a função e os armazena em um dicionário.

```
def exemplo(**kwargs):
    print(kwargs)

exemplo(nome="Ana", idade=25)
```



```
{'nome': 'Ana', 'idade': 25}
```

Argumentos

- A sintaxe especial **kwargs em definições de funções é usada para passar uma lista de argumentos de comprimento variável.
- O **kwargs coleta todos os keyword arguments adicionais passados para a função e os armazena em um dicionário.

```python
def exibir_informacoes(**kwargs):
    print("Argumentos recebidos:")
    for chave, valor in kwargs.items():
        print(f"{chave}: {valor}")

# Chamando a função com diferentes quantidades de argumentos
exibir_informacoes(nome="Ana")

print("-" * 30)

exibir_informacoes(nome="Carlos", idade=30, cidade="São Paulo")

print("-" * 30)

exibir_informacoes(produto="Notebook", preco=3500, garantia="2 anos", estoque=15)
```

```
Argumentos recebidos:
nome: Ana
------------------------------
Argumentos recebidos:
nome: Carlos
idade: 30
cidade: São Paulo
------------------------------
Argumentos recebidos:
produto: Notebook
preco: 3500
garantia: 2 anos
estoque: 15
```

ESCOPO

- Cada função define um escopo, ou seja, um novo espaço com códigos específicos.
- Todas as variáveis criadas no escopo definido na função são variáveis locais
  - Variáveis locais: existem apenas dentro da função onde são criadas.

- As variáveis na lista de argumentos da função também são variáveis locais.

- As variáveis na lista de argumentos permitem a troca de informações entre funções.

ESCOPO

```
#declaração da função

def f(x):
    a=b=c=x
    print(a,b,c)
    return a+b+c

a,b,c=1,2,3
#chamada da função
d=f(4)
print(d)
print(a,b,c)
```

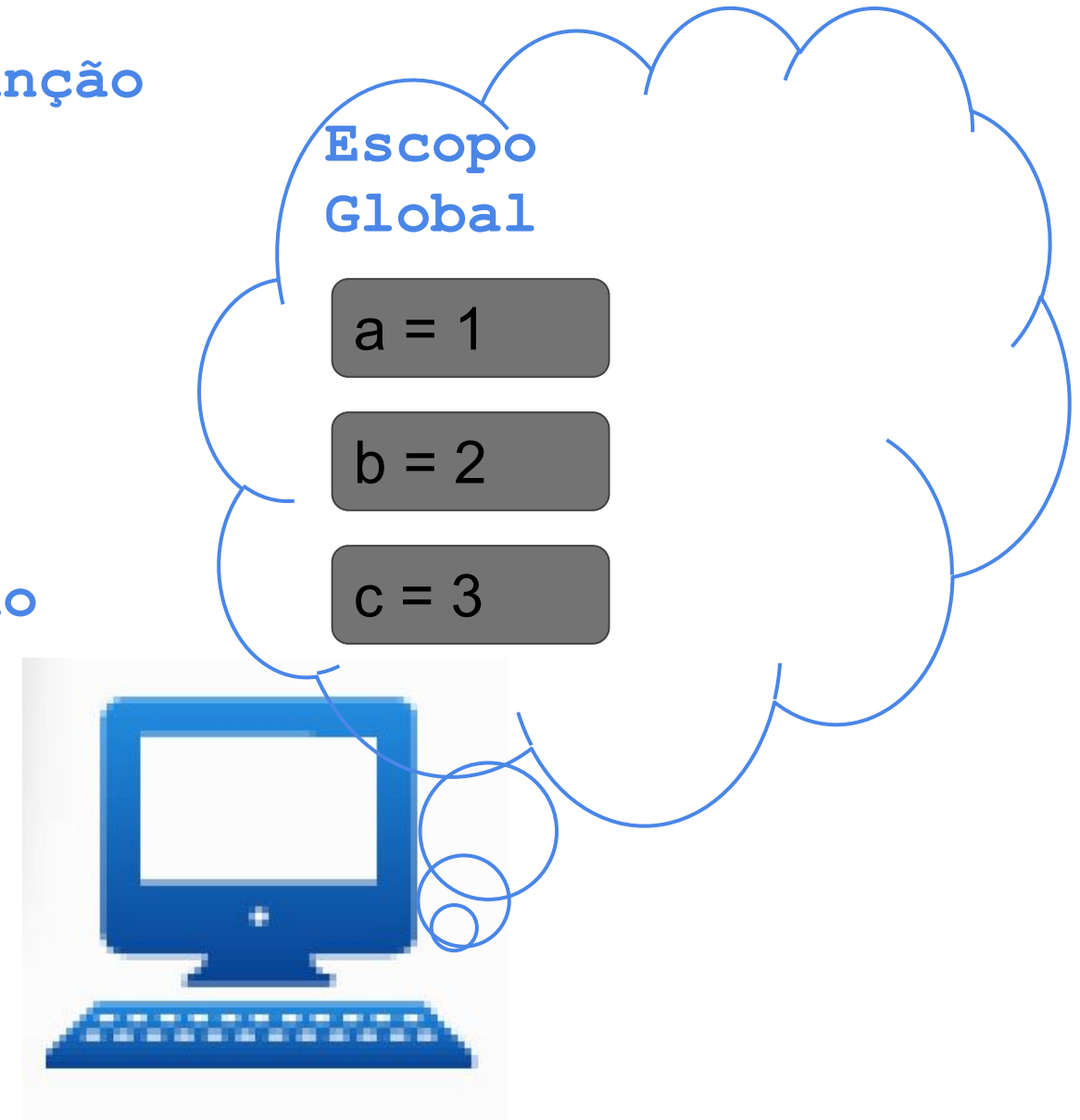


ESCOPO

```
#declaração da função

def f(x):
    a=b=c=a=x
    print(a,b,c)
    return a+b+c

a,b,c=1,2,3
#chamada da função
d=f(4)
print(d)
print(a,b,c)
```

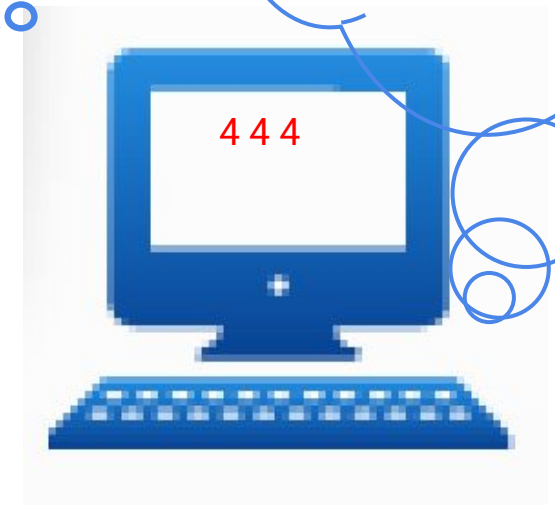


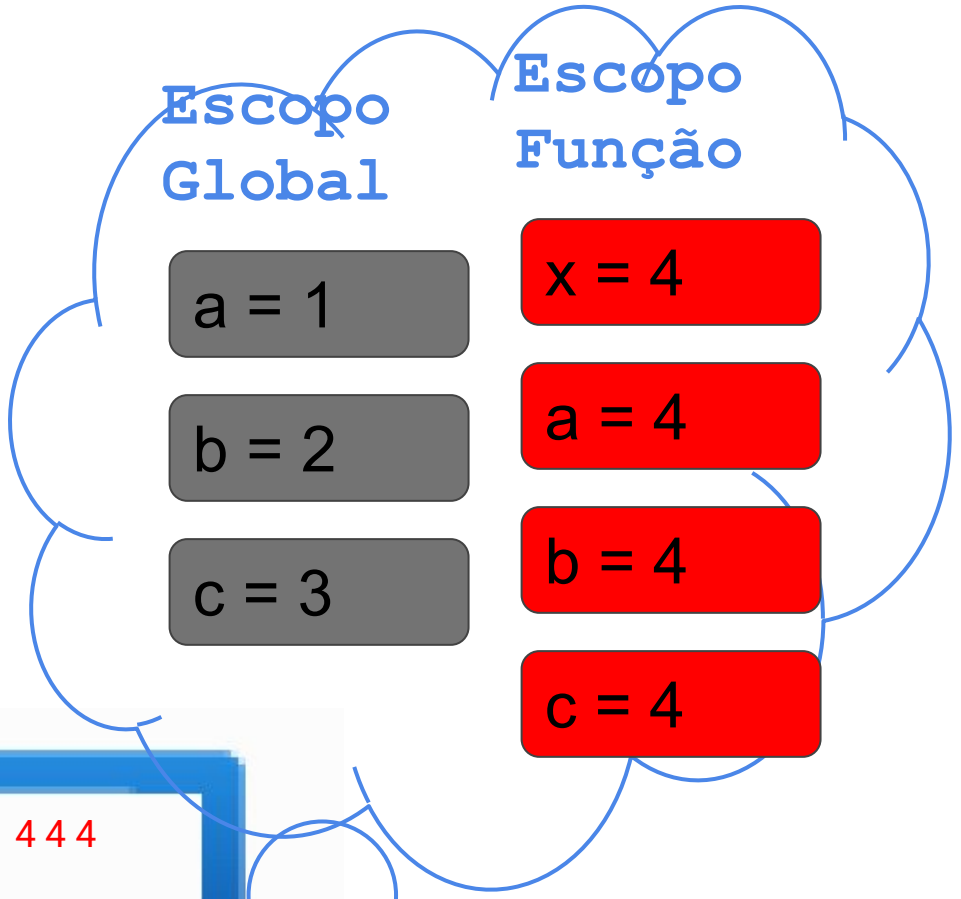
ESCOPO

```
#declaração da
função

def f(x):
    a=b=c=x
    print(a,b,c)
    return a+b+c

a,b,c=1,2,3
#chamada da função
d=f(4)
print(d)
print(a,b,c)
```



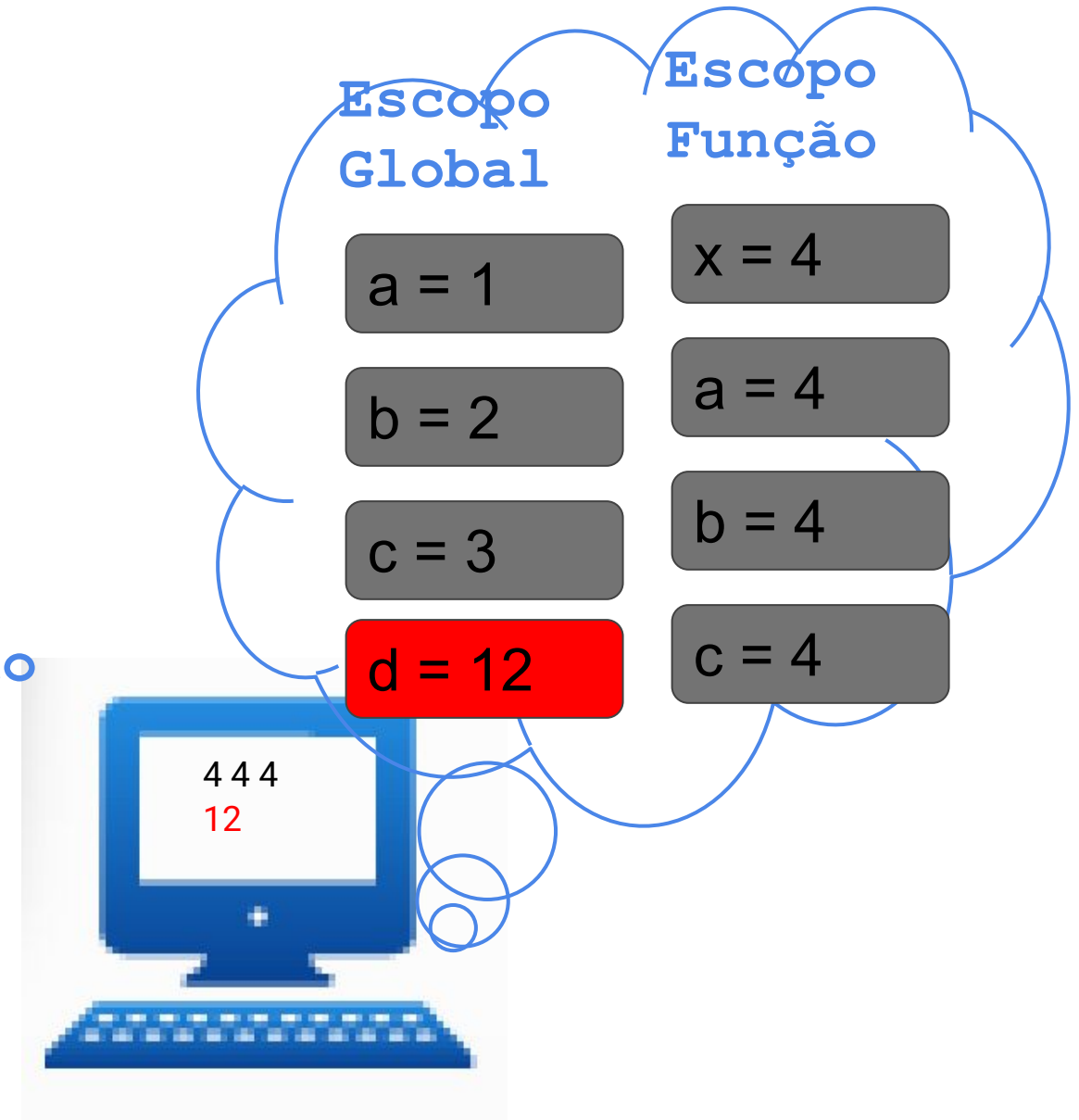


ESCOPO

```
#declaração da
função

def f(x):
    a=b=c=a=x
    print(a,b,c)
    return a+b+c

a,b,c=1,2,3
#chamada da função
d=f(4)
print(d)
print(a,b,c)
```

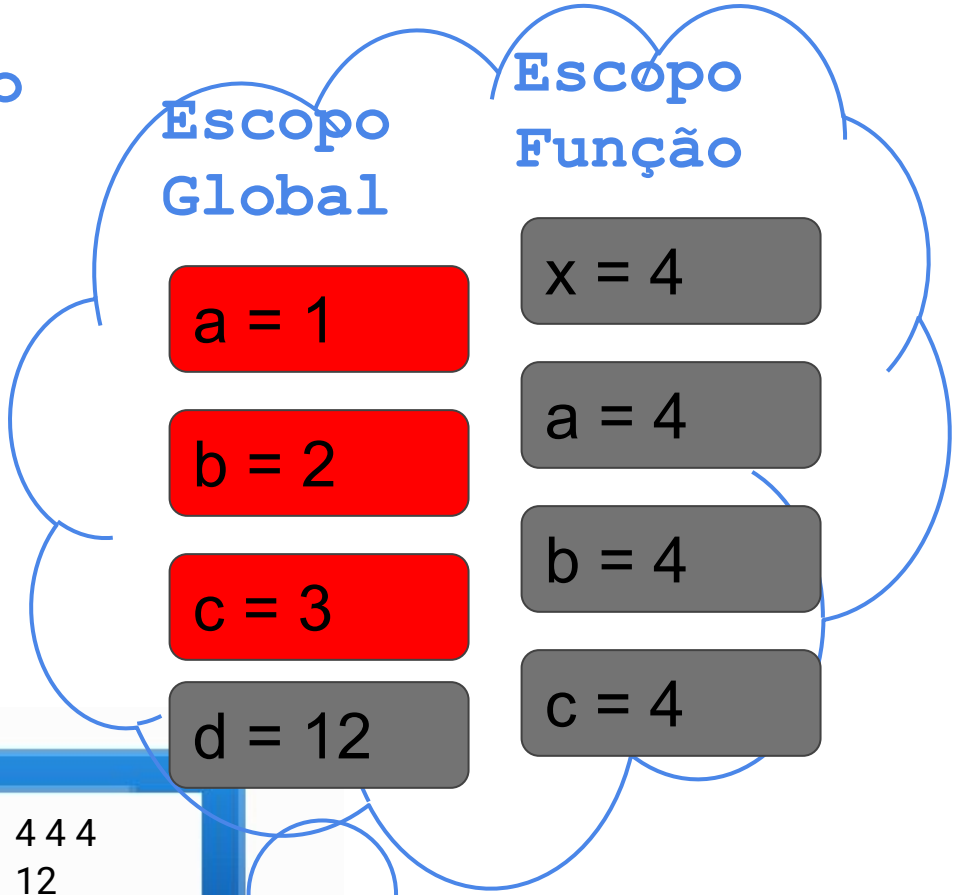


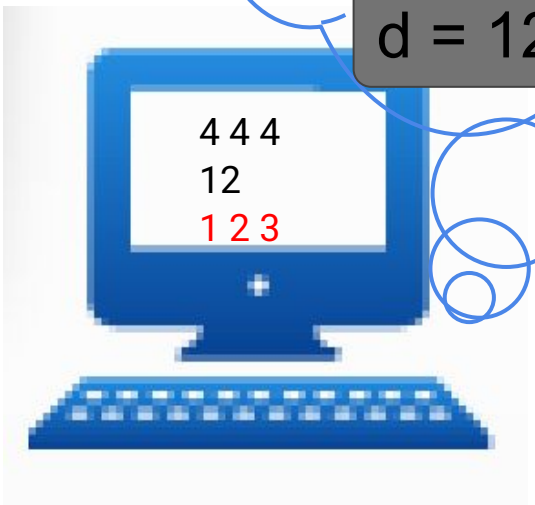
ESCOPO

```
#declaração da função

def f(x):
    a=b=c=a=x
    print(a,b,c)
    return a+b+c

a,b,c=1,2,3
#chamada da função
d=f(4)
print(d)
print(a,b,c)
```



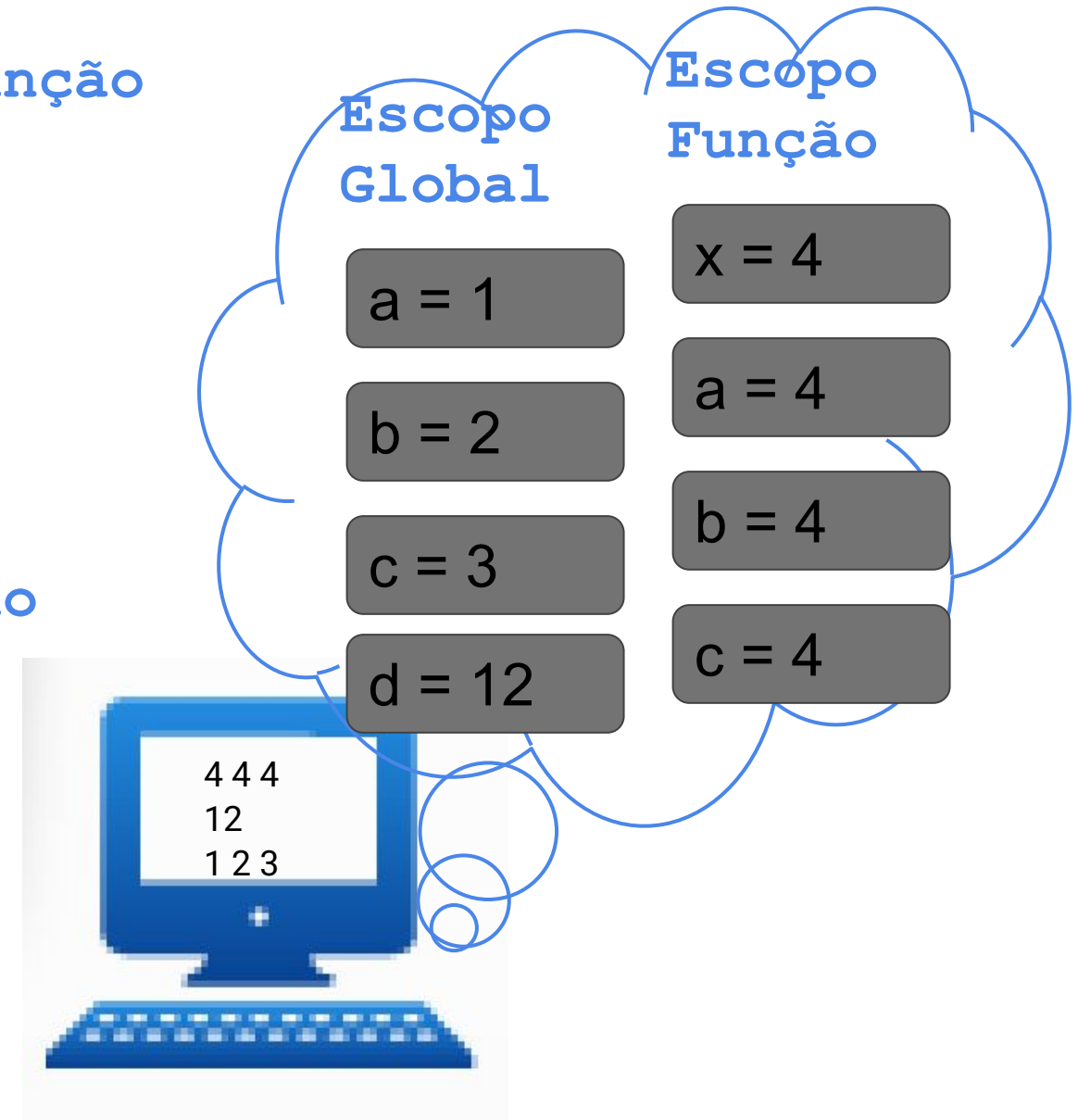


ESCOPO

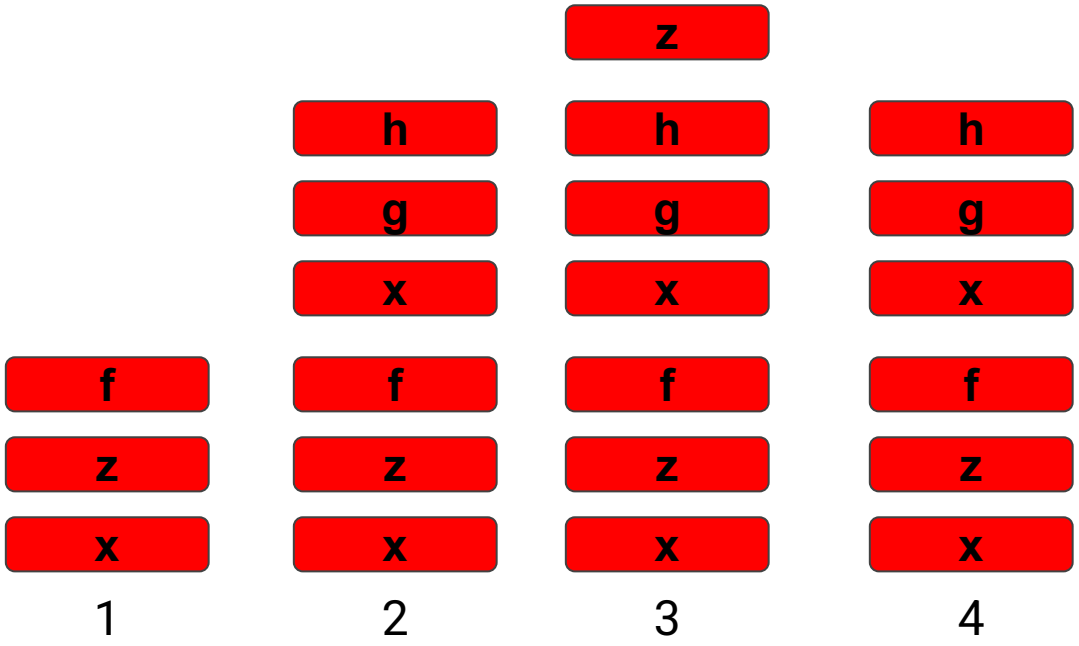
```
#declaração da função

def f(x):
    a=b=c=a=x
    print(a,b,c)
    return a+b+c

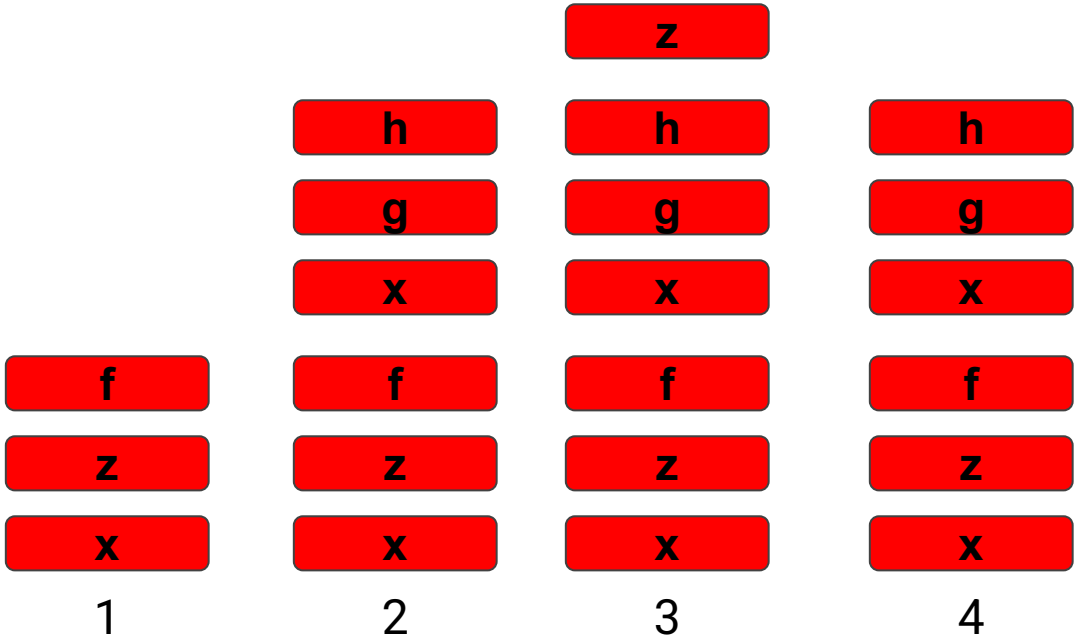

a,b,c=1,2,3
#chamada da função
d=f(4)
print(d)
print(a,b,c)
```

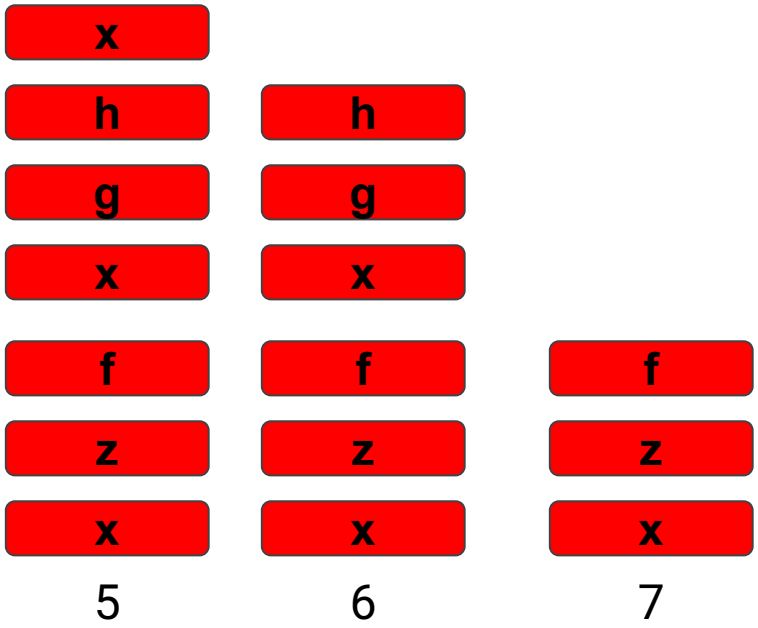


ESCOPO




```
#Exemplo: Escopo -
Empilhando chamadas
def f(x):
 def g():
   x = 'abc'
   print(f'{x =}')
 def h():
   z = x
   print(f'{z =}')
 x = x + 1
 print(f'{x =}')
 h()
 g()
 print(f'{x =}')
 return g
x = 3
z = f(x)
print(f'{x =}')
x
print(f'{z =}')
z()
```






```
#Exemplo: Escopo -
Empilhando chamadas
def f(x):
 def g():
  x = 'abc'
  print(f'{x =}')
 def h():
    z = x
    print(f'{z =}')
 x = x + 1
 print(f'{x =}')
 h()
 g()
 print(f'{x =}')
 return g
x = 3
z = f(x)
print(f'{x =}')
x
print(f'{z =}')
z()
```

Exemplo

Exemplos

Modularização

```python
def encontra_raiz(x,pot,epsilon):
    if x<0 and pot%2==0:
        return None
    minimo = min(-1,x)
    maximo = max(1,x)
    # método_bisseccao
    raiz = (minimo + maximo)/2
    while abs(raiz**pot - x) >= epsilon:
        if raiz**pot < x:
            minimo = raiz
        else:
            maximo = raiz
        raiz = (minimo + maximo)/2
    return raiz
```

```python
def encontra_raiz_limites(x,pot):
    '''x é real, pot é inteiro
        retorna minino e maximo tal que
        mininmo^pot <= x <= maximo^pot'''
    minimo = min(-1,x)
    maximo = max(1,x)
    return minimo, maximo

def metodo_bisseccao(x,pot,episilon,minimo,maximo):
    ''' x, episilon, minimo, maximo são reais
        epsilon > 0, minimo < maximo
        retorna raiz tal que
        abs(raiz**pot - x) < episilon'''
    raiz = (minimo + maximo)/2
    while abs(raiz**pot - x) >= episilon:
        if raiz**pot < x:
            minimo = raiz
        else:
            maximo = raiz
        raiz = (minimo + maximo)/2
    return raiz

def encontra_raiz(x,pot,epsilon):
    '''x é inteiro ou real, epsilon é float,
    pot é inteiro, epsilon >0 e pot >=1,
    retorna raiz de x com epsilon de precisão'''
    minimo, maximo = encontra_raiz_limites(x,pot)
    return metodo_bisseccao(x,pot,epsilon,minimo,maximo)

encontra_raiz(25,2,0.001)
help(encontra_raiz_limites)
help(metodo_bisseccao)
help(encontra_raiz)
```

Modularização

```
encontra_raiz(25,2,0.001)=4.9999237060546875
Help on function encontra_raiz_limites in module __main__:

encontra_raiz_limites(x, pot)
    x é real, pot é inteiro
    retorna minino e maximo tal que
    mininmo^pot <= x <= maximo^pot

Help on function metodo_bisseccao in module __main__:

metodo_bisseccao(x, pot, episilon, minimo, maximo)
    x, epsilon, minimo, maximo são reais
    epsilon > 0, minimo < maximo
    retorna raiz tal que
    abs(raiz**pot - x) < epsilon

Help on function encontra_raiz in module __main__:

encontra_raiz(x, pot, epsilon)
    x é inteiro ou real, epsilon é float,
    pot é inteiro, epsilon >0 e pot >=1,
    retorna raiz de x com epsilon de precisão
```

```
1  def encontra_raiz_limites(x,pot):
2      '''x é real, pot é inteiro
3          retorna minino e maximo tal que
4          mininmo^pot <= x <= maximo^pot'''
5      minimo = min(-1,x)
6      maximo = max(1,x)
7      return minimo, maximo
8
9  def metodo_bisseccao(x,pot,episilon,minimo,maximo):
10     ''' x, episilon, minimo, maximo são reais
11         epsilon > 0, minimo < maximo
12         retorna raiz tal que
13         abs(raiz**pot - x) < episilon'''
14     raiz = (minimo + maximo)/2
15     while abs(raiz**pot - x) >= episilon:
16         if raiz**pot < x:
17             minimo = raiz
18         else:
19             maximo = raiz
20         raiz = (minimo + maximo)/2
21     return raiz
22
23 def encontra_raiz(x,pot,epsilon):
24     '''x é inteiro ou real, epsilon é float,
25     pot é inteiro, epsilon >0 e pot >=1,
26     retorna raiz de x com epsilon de precisão'''
27     minimo, maximo = encontra_raiz_limites(x,pot)
28     return metodo_bisseccao(x,pot,epsilon,minimo,maximo)
29
30 encontra_raiz(25,2,0.001)
31 help(encontra_raiz_limites)
32 help(metodo_bisseccao)
33 help(encontra_raiz)
```

Função Lambda

- São funções anônimas em Python, escritas em uma única linha.

lambda argumentos : expressão
palavra-chave lambda.

- Alternativa rápida a funções curtas usando def.

```
# Função
def soma(a, b):
    return a + b

# Função lambda equivalente
soma_lambda = lambda a, b: a + b

print(soma(2, 3))        # Saída: 5
print(soma_lambda(2, 3)) # Saída: 5
```

Função Lambda

```
alunos = [("Ana", 22), ("Carlos", 20), ("Beatriz", 21)]

# Ordenar por idade
ordenados = sorted(alunos, key=lambda x: x[1])
print(ordenados)
```

```
[('Carlos', 20), ('Beatriz', 21), ('Ana', 22)]
```

A função lambda retorna x[1] (a idade), usada como critério de ordenação.

Função Lambda

A função como argumento

```
def avalia_raiz(raiz):
  return  raiz**2


def metodo_bisseccao(x,avalia_raiz,epsilon,minimo,maximo):
  ''' x, epsilon, minimo, maximo são reais
      epsilon > 0, minimo < maximo
      retorna raiz tal que
      abs(raiz**pot - x) < epsilon'''
  raiz = (minimo + maximo)/2
  while abs(avalia_raiz(raiz) - x) >= epsilon:
    if avalia_raiz(raiz) < x:
      minimo = raiz
    else:
      maximo = raiz
    raiz = (minimo + maximo)/2
  return raiz

minimo,maximo=encontra_raiz_limites(99,2)
print(f'{metodo_bisseccao(99,avalia_raiz,0.001,minimo,maximo)=}')
```

A função lambda como argumento

```
def metodo_bisseccao(x,avalia_raiz,epsilon,minimo,maximo):
  ''' x, epsilon, minimo, maximo são reais
      epsilon > 0, minimo < maximo
      retorna raiz tal que
      abs(raiz**pot - x) < epsilon'''
  raiz = (minimo + maximo)/2
  while abs(avalia_raiz(raiz) - x) >= epsilon:
    if avalia_raiz(raiz) < x:
      minimo = raiz
    else:
      maximo = raiz
    raiz = (minimo + maximo)/2
  return raiz

minimo,maximo=encontra_raiz_limites(99,2)
print(f'{metodo_bisseccao(99,lambda x: x**2,0.001,minimo,maximo)=}')
```

Função Lambda

```python
def log(x,base,epsilon):
    def find_log_bounds(x,base):
        maximo = 0
        while base**maximo < x:
            maximo += 1
        minimo = maximo - 1
        return minimo, maximo
    minimo, maximo = find_log_bounds(x,base)
    return metodo_bisseccao(x,lambda y: base**y,epsilon,minimo,maximo)

print(f'{log(99,10,0.001)=}')
```

```python
def avalia_raiz_log(base):
    return  lambda y: base**y

def find_log_bounds(x,base):
    maximo = 0
    while base**maximo < x:
        maximo += 1
    minimo = maximo - 1
    return minimo, maximo

def log(x,base,epsilon):
    minimo, maximo = find_log_bounds(x,base)
    avalia_raiz = avalia_raiz_log(base)
    return metodo_bisseccao(x,avalia_raiz,epsilon,minimo,maximo)

print(f'{log(99,10,0.001)=}')
```

Recursão

- Uma função é recursiva quando chama a si mesma.
- Exemplo:

```
def sum(n):
  if (n<=1):
     return n
  return (n+sum(n-1))
```

Entrada	Saída	
Sum(1)	1	
Sum(2)	2 + sum(1)	2+ 1
Sum(3)	3 + sum(2)	3 + 2 + 1
Sum(4)	4 + sum(3)	4 + 3 + 2 + 1

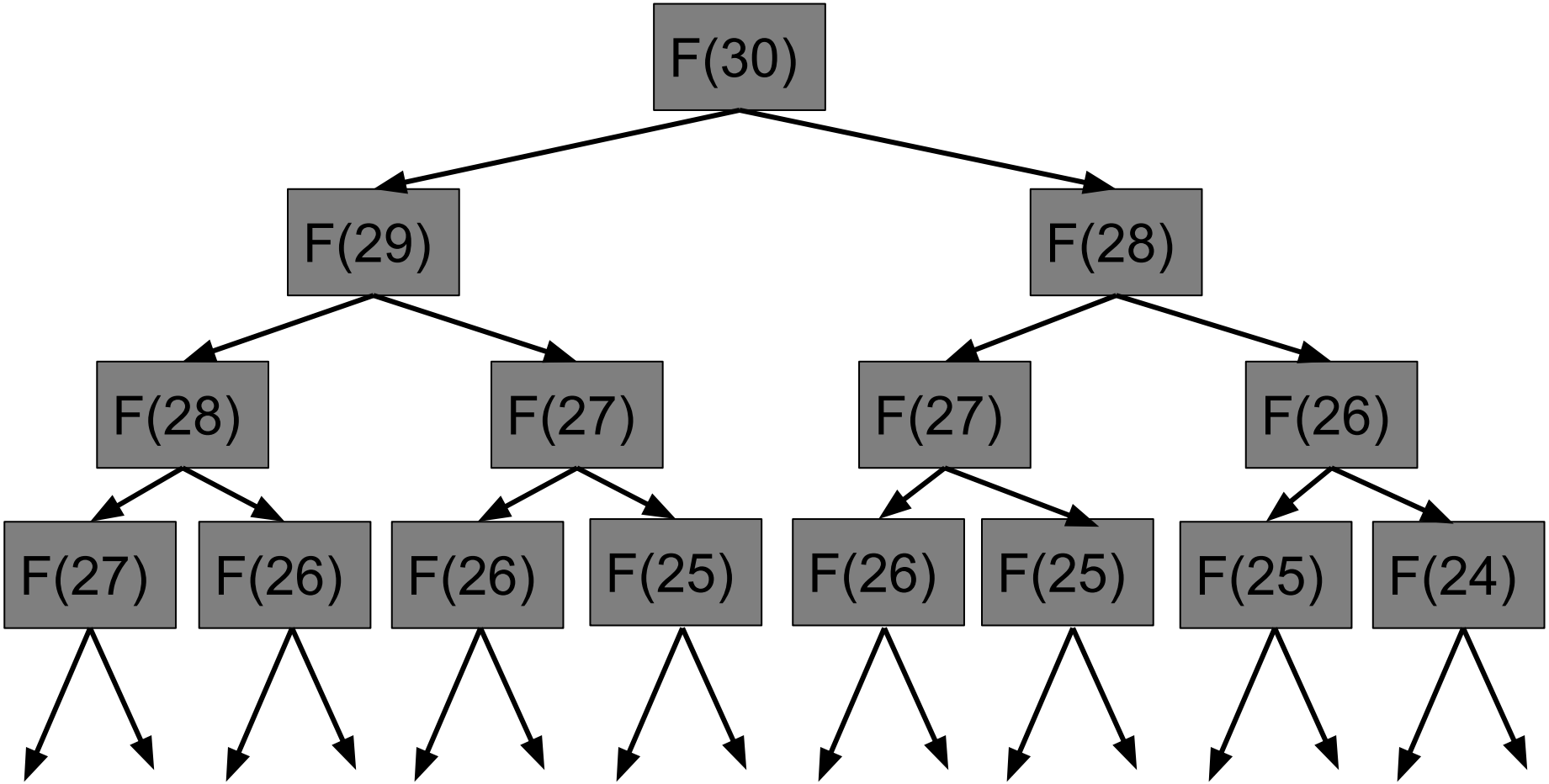
Recursão

- Exemplo: Sequência de Fibonacci.
  f(0)=0, f(1)=1, f(i+1)=f(i)+f(i-1), i=1,2,...

```
def fib(int n):
   if(n<=1):
      return 1
   else:
      return (fib(n-1) + fib(n-2))
```

n	F(n)	Número de chamadas da função
0	1	1
1	1	1
2	1	3
...	...	...
23	28657	92735
24	46368	150049

Recursão



Recursão

- Recursão vs Iteração
  - Iteração usa estrutura de repetição
  - Recursão usa estrutura de seleção (chamadas de funções repetitivas).
  - Iteração usa violação da condição como critério de parada.
  - Recursão usa passo base como critério de parada.
  - Ambos podem ser executados infinitamente, se o critério de parada não for satisfeito.
- Desvantagens da Recursão
  - Gera sobrecarga (overhead) com as chamadas de função, gerando gasto de tempo de processamento e espaço de memória.
  - Uma cópia da função (variáveis da função) é criada, consumindo memória.
  - Logo, a iteração tende a ser mais rápida por não fazer repetidas chamadas de funções.

Recursão

- Quando usar Recursão ou Iteração?

Recursão	Iteração
Estruturas condicionais	Estruturas de repetição
Repetição implícita	Repetição explícita
Caso base como critério de parada	Condição com critério de parada
Lento	Rápido
Solução simples	Solução complexa
Fácil manutenção	Difícil Manutenção

Recursão

Palíndromo

ANA

Arara rara

Anotaram a data da maratona

Exemplos